

GRADO EN INGENIERÍA INFORMÁTICA DE GESTIÓN Y SISTEMAS DE INFORMACIÓN

TRABAJO DE FIN DE GRADO

**CONVERSIÓN DE ACTAS DEL AYUNTAMIENTO DE
BILBAO A KNOWLEDGE GRAPHS (RDF, OWL,
SPARQL, SHACL) Y VECTORIAL MEDIANTE LLMS,
ENLAZADO CON WIKIDATA Y OFRECIENDO
FUNCIONALIDAD RAG. INTERFAZ WEB: VISUAL,
CONSULTAS, CONVERSACIONAL**



Alumno: Lopez, Sanchez, Peio

Director: Egaña Aranguren, Mikel

Curso: 2026-2027

Fecha: <XXXX, día, mes, año>

Resumen Laburpena Abstract

Resumen:

Este trabajo aborda la transformación de las actas del Pleno del Ayuntamiento de Bilbao (2007–2026), publicadas en formato PDF, en dos sistemas complementarios de recuperación de información consultables en lenguaje natural: un RAG vectorial y un sistema GraphRAG. A partir de la descarga automatizada del portal municipal, se extraen de forma estructurada las proposiciones individuales junto con sus metadatos esenciales: fecha de la sesión, proponente, tema, entidades, página exacta y el resultado de la votación –tanto agregado por grupo como nominal por concejal cuando consta en el acta–. Esta información nutre un grafo de conocimiento RDF/OWL que modela la actividad plenaria, combinando extracción determinista para garantizar la fidelidad de los votos con modelos de lenguaje para la clasificación temática y el reconocimiento de entidades.

Sobre este corpus de 3.422 proposiciones se implementan dos arquitecturas de consulta: un RAG vectorial con búsqueda híbrida multicanal y reordenación semántica (reranking), y un sistema GraphRAG que traduce el lenguaje natural a consultas SPARQL para resolver preguntas de agregación y conteo mediante razonamiento automático. Ambos sistemas garantizan la trazabilidad citando la página exacta del documento original y se integran en una interfaz conversacional construida con Chainlit. El trabajo concluye con una evaluación comparativa sistemática sobre un conjunto de preguntas de referencia contrastadas exhaustivamente contra los documentos oficiales.

Palabras Clave: GraphRAG, RAG vectorial, grafo de conocimiento, SPARQL, LLM, datos abiertos municipales.

Laburpena:

Lan honek Bilboko Udalaren Osoko Bilkuraren aktak (2007–2026), PDF formatuan argitaratuak, hizkuntza naturalean kontsultatu daitezkeen informazio-berreskurapeneko bi sistema osagarritan –RAG bektorial bat eta GraphRAG sistema bat– eraldatzeari ekiten dio. Udalaren atarian eskuratutako aktetatik, egiturazko moduan ateratzen dira proposamen indibidualak, beren funtsezko metadatuarekin batera: bilkuraren data, proposatzailea, gaia, entitateak, orrialde zehatza eta bozketaren emaitza –taldeka metatuta zein, aktan hala jasotzen denean, zinegotziz zinegotzi–. Informazio horrek RDF/OWL ezagutza-grafo bat elikatzen du, osoko bilkuraren jardura modelatzen duena, bozen zehaztasuna bermatzeko atzemate determinista eta gai-sailkapenerako eta entitateen ezagutzarako hizkuntza-ereduak konbinatuz.

3.422 proposamenez osatutako corpus horren gainean, kontsulta-arkitektura bi ezartzen dira: bilaketa hibrido anizkanaleko eta birsailkapen semantikoko (reranking) RAG bektorial bat, eta hizkuntza naturala SPARQL kontsultetara itzultzen duen GraphRAG sistema bat, arazoiketa automatikoaren bidez metaketa- eta zenbaketa-galderak ebazteko. Bi sistemek

jatorrizko dokumentuaren orrialde zehatza aipatuz bermatzen dute trazabilitatea, eta Chainlit-ekin eraikitako elkarrikketa-interfaze batean integratzen dira. Lanak jatorrizko dokumentuen aurka zorrotz egiaztatutako erreferentzia-galdera multzo baten gaineko ebaluazio konparatibo sistematiko batekin ixten du.

Gako-hitzak: GraphRAG, RAG bektoriala, ezagutza-grafoa, SPARQL, LLM, udal datu irekiak.

Abstract:

This work addresses the transformation of the minutes of the Plenary Session of the Bilbao City Council (2007–2026), published in PDF format, into two complementary natural-language information retrieval systems: a vector RAG and a GraphRAG system. From documents downloaded automatically from the municipal portal, individual proposals are extracted in structured form together with their essential metadata: session date, proposer, topic, entities, exact page, and the outcome of the vote –both aggregated by political group and, when explicitly recorded in the minutes, nominal by councillor–. This information feeds an RDF/OWL knowledge graph that models plenary activity, combining deterministic extraction to guarantee the fidelity of the votes with language models for topic classification and entity recognition.

Two query architectures are implemented over this corpus of 3,422 proposals: a vector RAG with multi-channel hybrid search and semantic reranking, and a GraphRAG system that translates natural language into SPARQL queries to resolve aggregation and counting questions through automated reasoning. Both systems guarantee traceability by citing the exact page of the original document and are exposed through a conversational web interface built with Chainlit. The work concludes with a systematic comparative evaluation over a set of reference questions verified exhaustively against the original documents.

Keywords: GraphRAG, vector RAG, knowledge graph, SPARQL, LLM, open municipal data.

Índice

Resumen Laburpena Abstract	1
Lista de figuras	6
Lista de tablas	7
Lista de acrónimos	8
1. Introducción	9
1.1. Motivación	9
1.2. Objetivos y alcance	10
1.3. Conexión con los Objetivos de Desarrollo Sostenible	11
1.4. Estructura de la memoria	12
2. Contexto	13
2.1. Transparencia y digitalización de la administración pública	13
2.2. El auge de los LLMs	13
2.3. RAG	14
2.4. Grafos de Conocimiento	15
2.5. GraphRAG	15
2.6. El Pleno del Ayuntamiento de Bilbao	15
3. Descripción de requerimientos y análisis del estado del arte	17
3.1. Requisitos del sistema	17
3.2. Estado del arte de las tecnologías utilizadas	17
4. Planificación del proyecto	18
4.1. Diagrama EDT	18

4.2. Descripción de tareas	18
4.3. Gestión del tiempo	23
4.4. Metodología de desarrollo	23
4.5. Evaluación económica	24
4.6. Análisis de riesgos	25
5. Análisis de alternativas	26
5.1. Modelos de lenguaje: local (Ollama) frente a API en la nube (Groq / Gemini)	26
5.2. Almacenamiento del grafo: rdflib/OWL-RL frente a una base de grafos dedicada (p.ej. GraphDB, Neo4j)	26
5.3. Almacén vectorial y recuperación: ChromaDB e híbrida (semántica + literal + temática) frente a solo semántica	26
6. Descripción de la solución propuesta	27
6.1. Arquitectura general del sistema	27
6.2. Construcción del grafo de conocimiento	27
6.3. Sistema de recuperación de información	27
6.4. Generación de respuestas	27
6.5. Interfaz web del sistema	27
6.6. Flujo completo del sistema	27
6.7. Configuración y decisiones de implementación	27
7. Corpus y preprocesado de datos	28
7.1. El corpus de actas del Pleno	28
7.2. Pipeline de preprocesado	28
8. Análisis de resultados	29
8.1. Objetivo de la evaluación	29
8.2. Metodología	29
8.3. Análisis sobre la recuperación	29
8.4. Análisis sobre la generación	29
8.5. Comparativa GraphRAG frente a RAG vectorial	29
8.6. Configuración final y valoración global	29
8.7. Limitaciones	29
9. Conclusiones y trabajo futuro	30

9.1. Conclusiones del trabajo	30
9.2. Cumplimiento de objetivos	30
9.3. Conclusiones sobre las pruebas realizadas	30
9.4. Contribuciones científico-técnicas	30
9.5. Trabajo futuro	30
Bibliografía	31

Lista de figuras

1.	Icono del Objetivo 16: Paz, justicia e instituciones sólidas	12
2.	Icono del Objetivo 11: Ciudades y comunidades sostenibles	12
3.	Fachada del Ayuntamiento de Bilbao	16
4.	Diagrama EDT del proyecto	18

Lista de tablas

1.	Paquete de trabajo: 1. Gestión y planificación	19
2.	Paquete de trabajo: 1.1. Reuniones con el director	19
3.	Paquete de trabajo: 1.2. Gestión del tiempo	19
4.	Paquete de trabajo: 2. Investigación y estudio previo	19
5.	Paquete de trabajo: 2.1. Estudio del estado del arte	19
6.	Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas	20
7.	Paquete de trabajo: 3. Desarrollo del sistema	20
8.	Paquete de trabajo: 3.1. Extracción y preprocesado del corpus	20
9.	Paquete de trabajo: 3.2. RAG vectorial	20
10.	Paquete de trabajo: 3.3. GraphRAG	21
11.	Paquete de trabajo: 3.4. Interfaz web	21
12.	Paquete de trabajo: 4. Evaluación del sistema	21
13.	Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia . .	21
14.	Paquete de trabajo: 4.2. Evaluaciones y comparativa	22
15.	Paquete de trabajo: 4.3. Validación contra PDF real y regresión	22
16.	Paquete de trabajo: 5. Documentación	22
17.	Paquete de trabajo: 5.1. Redacción de la memoria	23
18.	Paquete de trabajo: 5.2. Documentación técnica	23
19.	Coste de mano de obra	24
20.	Coste de hardware (amortizado)	24
21.	Coste de software	24
22.	Costes indirectos (electricidad)	25
23.	Coste total del proyecto	25
24.	Riesgo: 1. Dependencia de APIs externas	25
25.	Categorías de preguntas de la batería de referencia (ejemplo)	29

Lista de acrónimos

LLM Large Language Model

RAG Retrieval Augmented Generation

GraphRAG Graph Retrieval Augmented Generation

RDF Resource Description Framework

OWL Web Ontology Language

SPARQL SPARQL Protocol and RDF Query Language

SKOS Simple Knowledge Organization System

SHACL Shapes Constraint Language

LOD Linked Open Data

API Application Programming Interface

PDF Portable Document Format

OCR Optical Character Recognition

TFG Trabajo Fin de Grado

1. Introducción

El portal de actas del Ayuntamiento de Bilbao publica, para cada sesión del Pleno desde 2007 hasta la actualidad, varios documentos PDF distintos: el orden del día, extractos, el resumen de la sesión y el acta completa. De estos, el acta es el único que recoge de forma íntegra las proposiciones, el debate y el resultado de cada votación, por lo que es el documento que utiliza este trabajo; cada acta puede superar el centenar de páginas, y el corpus completo reúne cerca de veinte años de actas y 3.422 proposiciones distintas.

Este capítulo presenta la motivación que ha llevado al desarrollo del proyecto, los objetivos planteados, su relación con los Objetivos de Desarrollo Sostenible y la estructura del resto de la memoria.

1.1. Motivación

Dentro de cada acta, el contenido relevante –las proposiciones y sus votaciones– se mezcla con una cantidad considerable de contenido puramente procedimental: listas de asistentes, fórmulas de quórum, aprobación del acta anterior, cuestiones de orden o informes que solo se dan por conocidos sin votación. Identificar qué parte de cada acta es efectivamente una proposición –y separarla del resto– es en sí mismo parte del problema a resolver.

A esto se suma un problema práctico adicional: encontrar información concreta –qué votó un grupo político sobre un tema, cuántas proposiciones sobre vivienda se presentaron en un año determinado, cómo ha evolucionado un asunto a lo largo de los mandatos– exige leer manualmente decenas de documentos extensos, o como mínimo saber de antemano en qué fecha y página buscar. No existe ninguna vía para hacer preguntas en lenguaje natural sobre este archivo y obtener una respuesta verificable, con cifras y citas a la fuente original.

1.1.1. A quién beneficia el sistema

El sistema desarrollado tiene utilidad directa para varios perfiles distintos de usuario, cada uno con una forma de preguntar y un tipo de respuesta que le interesa más:

Ciudadanía Puede preguntar en lenguaje natural por el estado o el resultado de una propuesta concreta –p. ej. “¿qué se votó sobre el polideportivo de mi barrio?”– sin necesidad de conocer de antemano en qué acta ni en qué página buscar, y obtener la cita exacta al documento original como respaldo.

Periodistas Se benefician sobre todo de las preguntas de agregación y comparación entre

grupos políticos –p. ej. cuántas propuestas sobre vivienda ha presentado cada partido en un mandato, o cómo ha votado cada grupo un asunto concreto–, el tipo de consulta en el que el sistema GraphRAG aporta más valor frente a una búsqueda de texto convencional (Capítulo 2).

Personal técnico municipal Puede recuperar rápidamente el precedente de un asunto ya tratado en plenos anteriores –qué se acordó, qué grupo lo propuso, si se aprobó con enmienda– sin depender de la memoria de quien estuvo presente en aquella sesión ni de una búsqueda manual en el archivo.

Investigadores Disponen de un corpus estructurado y consultable de forma sistemática –por tema, por grupo, por periodo temporal– que permite estudiar la actividad plenaria a lo largo de casi veinte años sin procesar manualmente cada acta en PDF.

1.2. Objetivos y alcance

El objetivo general de este TFG es cerrar la brecha descrita en la motivación: construir un sistema que permita consultar en castellano, mediante preguntas formuladas de forma natural, el contenido íntegro de las actas del Pleno, y que responda con datos verificables –enlazados a la página exacta del PDF de origen– en lugar de con texto generado sin respaldo documental.

1.2.1. Objetivos específicos

Del objetivo general se derivan los siguientes objetivos específicos:

1. Extraer de forma automatizada las proposiciones individuales de cada acta – junto con su fecha, grupo o particular proponente, tema, entidades mencionadas y resultado de la votación– a partir del corpus de PDF publicado por el Ayuntamiento (Capítulo 7).
2. Construir un grafo de conocimiento RDF/OWL que modele las proposiciones, los grupos políticos, los concejales y los temas, y que permita razonamiento automático sobre relaciones temáticas jerárquicas (Capítulo 6).
3. Implementar un sistema de recuperación de información basado en búsqueda vectorial híbrida (semántica, literal y temática) sobre el mismo corpus.
4. Implementar un segundo sistema de recuperación, GraphRAG, que traduzca la pregunta del usuario a una consulta SPARQL y la ejecute sobre el grafo de conocimiento, para responder preguntas de agregación y razonamiento multi-salto que el sistema vectorial no puede resolver con precisión.
5. Exponer ambos sistemas mediante una interfaz web conversacional que cite, para cada dato mostrado, el documento y la página exacta del PDF de origen.
6. Evaluar de forma comparativa ambos sistemas sobre un conjunto de preguntas de referencia con respuesta verificada a mano contra las actas originales, para caracterizar en qué tipo de preguntas destaca cada arquitectura (Capítulo 8).

1.2.2. Alcance del sistema desarrollado

El sistema cubre el corpus completo de actas del Pleno publicadas por el Ayuntamiento de Bilbao entre 2007 y 2026 (3.422 proposiciones), con extracción de metadatos, clasificación temática, dos sistemas de recuperación de información independientes y una interfaz web conversacional común a ambos.

Quedan fuera del alcance de esta entrega:

- El enlazado del grafo de conocimiento con Wikidata para desambiguar y enriquecer entidades no personales (empresas, lugares, organismos). Se valoró como extensión natural del grafo RDF/OWL, pero se descartó para esta entrega por el coste de desambiguación fiable que exige –en particular, distinguir entidades homónimas sin introducir enlaces incorrectos– y queda recogido como trabajo futuro (Capítulo 9).
- La desambiguación por género de concejales homónimos en la capa de voto nominal, más allá de la resolución por apellido ya implementada (Capítulo 7).
- Una evaluación con usuarios reales del sistema: la evaluación de este trabajo es exclusivamente técnica, sobre una batería de preguntas de referencia verificada contra las actas originales (Capítulo 8), sin sesiones de uso con los perfiles reales identificados en la Sección 1.1.1.

1.3. Conexión con los Objetivos de Desarrollo Sostenible

Con el objetivo de conseguir un futuro sostenible en el que se haya erradicado la desigualdad, se proteja el medioambiente y se garantice el bienestar de todos, los estados miembros de las Naciones Unidas aprobaron 17 objetivos el 25 de septiembre de 2015, con el propósito de cumplirlos para el año 2030.

El Objetivo de Desarrollo Sostenible (ODS) 16, *Paz, justicia e instituciones sólidas*, busca promover sociedades pacíficas e inclusivas, facilitar el acceso a la justicia para todos y construir instituciones eficaces, responsables e inclusivas a todos los niveles. En particular, su meta 16.10 persigue garantizar el acceso público a la información y proteger las libertades fundamentales, de acuerdo con la legislación nacional y los acuerdos internacionales.

El presente TFG se alinea con este objetivo al desarrollar una herramienta que convierte un archivo público municipal –las actas del Pleno del Ayuntamiento de Bilbao– en una fuente consultable en lenguaje natural, con cifras y citas verificables a la fuente original. Aunque las actas ya son públicas, su volumen (cerca de veinte años, más de un centenar de páginas por sesión) y su formato PDF no estructurado las hacen, en la práctica, de difícil acceso para quien no dispone del tiempo o la pericia necesarios para revisarlas manualmente. El sistema desarrollado reduce esa asimetría de información y facilita el escrutinio de la actividad municipal por parte de la ciudadanía, la prensa y la investigación, tal como se detalla en la Sección 1.1.1.

En segundo término, el trabajo conecta también con el ODS 11, *Ciudades y comunidades sostenibles*, cuya meta 11.3 promueve una urbanización inclusiva y participativa. Un acceso más directo al historial de decisiones del Pleno –qué se propuso, quién lo propuso y qué se decidió– es una condición previa para que la ciudadanía pueda participar de forma informada en los asuntos municipales, más allá del mero acceso a la información que cubre el ODS 16.



Figura 1: Icono del Objetivo 16: Paz, justicia e instituciones sólidas



Figura 2: Icono del Objetivo 11: Ciudades y comunidades sostenibles

1.4. Estructura de la memoria

Esta memoria se organiza como sigue. El Capítulo 2 introduce los conceptos técnicos en los que se apoya el sistema (modelos de lenguaje, RAG, grafos de conocimiento y GraphRAG) y describe el entorno institucional del Pleno del Ayuntamiento de Bilbao. El Capítulo 3 recoge los requisitos del sistema y el estado del arte de las tecnologías empleadas. El Capítulo 4 recoge la planificación temporal seguida, la evaluación económica y el análisis de riesgos del proyecto. El Capítulo 5 analiza las principales alternativas tecnológicas evaluadas antes de fijar la arquitectura final. El Capítulo 6 describe en detalle la solución desarrollada: los dos sistemas de recuperación de información contruidos –uno basado en búsqueda vectorial y otro en un grafo de conocimiento RDF/OWL consultado por SPARQL– y la interfaz web conversacional que los expone al usuario. El Capítulo 7 describe el corpus de actas de partida y el pipeline que lo transforma en proposiciones estructuradas. El Capítulo 8 presenta la evaluación comparativa de ambos sistemas. La memoria cierra con las conclusiones y los anexos.

2. Contexto

En este capítulo se presenta el contexto en el que se enmarca el trabajo, se introducen los conceptos de LLM, RAG, grafos de conocimiento y GraphRAG, que constituyen la base técnica del sistema desarrollado, y se describe el entorno institucional –el Pleno del Ayuntamiento de Bilbao– sobre el que se aplica.

2.1. Transparencia y digitalización de la administración pública

La transparencia y el acceso a la información pública son, desde hace más de una década, un objetivo explícito de la administración española. La Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno, obliga a las administraciones públicas a publicar de forma activa la información relativa a su actividad institucional. En paralelo, el movimiento de *open government* promueve que esa información no solo se publique, sino que se haga accesible y reutilizable, en vez de limitarse a cumplir el mínimo legal de publicación.

En la práctica, sin embargo, publicar un documento no equivale a hacerlo accesible. Los portales de transparencia municipales publican de forma sistemática actas, órdenes del día y resúmenes de sesión en formato PDF, un formato pensado para la lectura secuencial y no para la consulta puntual ni el análisis agregado. Cuando el volumen de documentos publicados crece –como ocurre con casi veinte años de actas del Pleno de una ciudad como Bilbao– esa distancia entre “publicado” y “accesible” se hace más evidente: la información está disponible en sentido legal, pero encontrar un dato concreto dentro de ella exige un esfuerzo de búsqueda manual que en la práctica limita quién puede efectivamente usarla.

En este contexto, las técnicas de inteligencia artificial que permiten consultar documentación extensa en lenguaje natural –y en particular las técnicas RAG y GraphRAG que se presentan en las siguientes secciones– ofrecen una vía para cerrar esa distancia entre publicación y acceso real, aplicada aquí a un caso concreto de documentación administrativa pública.

2.2. El auge de los LLMs

Los modelos de lenguaje de gran tamaño, conocidos como LLM por sus siglas en inglés *Large Language Models*, son modelos de inteligencia artificial entrenados sobre corpus masivos de texto que, dada una secuencia de entrada, predicen la continuación más probable. Esta capacidad, combinada con un ajuste posterior para seguir instrucciones, les permite realizar tareas tan diversas como responder preguntas, resumir documentos o

mantener una conversación con el usuario a partir de una instrucción (*prompt*).

El interés generalizado por los LLM se intensificó a partir de noviembre de 2022, cuando OpenAI presentó ChatGPT, una interfaz conversacional que popularizó el uso de estos modelos entre el público general. Desde entonces, la oferta de LLM disponibles –tanto propietarios como de código abierto– se ha ampliado considerablemente, lo que permite hoy elegir entre modelos servidos como API en la nube o modelos más pequeños ejecutables en local.

Un LLM, sin embargo, no tiene por sí mismo acceso a información posterior a su entrenamiento ni a documentos privados o muy específicos –como las actas de un ayuntamiento concreto–, y tampoco garantiza que lo que genera sea correcto: puede producir afirmaciones plausibles pero falsas, fenómeno conocido como *alucinación*, especialmente relevante en un contexto donde se maneja información institucional que debe ser verificable. A diferencia de un escenario con documentación empresarial confidencial, en este trabajo las actas del Pleno son de por sí públicas, por lo que la motivación para complementar el LLM con una fuente documental externa no es la confidencialidad, sino la precisión y la trazabilidad de cada dato: la respuesta debe poder verificarse contra el documento original, algo que el conocimiento interno del modelo no puede ofrecer por sí solo. Ambas limitaciones –la falta de acceso a documentación específica y la falta de verificabilidad– son las que motivan la técnica que se presenta a continuación.

2.3. RAG

La generación aumentada por recuperación, conocida como RAG por sus siglas en inglés *Retrieval-Augmented Generation*, es una técnica que combina un LLM con un mecanismo de recuperación de información, con el objetivo de que el modelo genere respuestas apoyándose en información obtenida de una base de conocimiento externa en lugar de únicamente en lo aprendido durante su entrenamiento.

El funcionamiento de un sistema RAG puede dividirse en dos fases. En la fase de preparación se construye la base de conocimiento: los documentos se procesan y se dividen en fragmentos más pequeños (*chunks*), que se transforman en representaciones vectoriales (*embeddings*) –números que capturan el significado del texto, de forma que fragmentos semánticamente similares quedan cerca en el espacio vectorial– y se almacenan en una base de datos vectorial. En la fase de consulta, la pregunta del usuario se transforma también en un *embedding*, se compara con los almacenados para recuperar los fragmentos más relevantes, y estos se incorporan al *prompt* que recibe el LLM para generar la respuesta.

Esta arquitectura tiene dos ventajas directas para este proyecto: permite responder sobre un corpus privado y actualizable (las actas del Pleno) sin reentrenar ningún modelo, y permite citar la fuente exacta de cada afirmación, ya que se conoce qué fragmentos –y por tanto qué documento y página– sustentan la respuesta generada.

La variante más directa de RAG se conoce como *Naive RAG*, que sigue exactamente el esquema de dos fases descrito arriba sin ningún refinamiento adicional. Sobre esa base existen variantes más avanzadas: *Advanced RAG* introduce mejoras en las distintas fases del proceso –como la reformulación de la consulta o la reordenación de los fragmentos recuperados– sin cambiar la arquitectura de fondo; y otras aproximaciones sustituyen el almacén de fragmentos de texto por una base de conocimiento estructurada, como un grafo de conocimiento, dando lugar a GraphRAG (véanse las Secciones 2.4 y 2.5).

En este trabajo se implementan dos sistemas de recuperación sobre el mismo corpus, no uno solo: un RAG vectorial de tipo *Advanced RAG* –con recuperación híbrida y reordenación de resultados– y un sistema GraphRAG completo, con el fin de comparar ambos enfoques sobre las mismas preguntas (Capítulo 8).

2.4. Grafos de Conocimiento

Un grafo de conocimiento representa la información como una red de entidades (nodos) y relaciones entre ellas (aristas), en lugar de como texto libre. El estándar utilizado en este trabajo es RDF (*Resource Description Framework*), que modela cada hecho como una tripleta *sujeto-predicado-objeto* (por ejemplo, **proposición X** --presentadaPor-->Grupo PP). Sobre RDF se puede definir una ontología con OWL (*Web Ontology Language*) –las clases y relaciones válidas del dominio– y consultar el grafo con el lenguaje SPARQL.

Una ventaja de este modelo frente al texto libre es el razonamiento automático: un motor de inferencia (*reasoner*) puede derivar hechos nuevos que no están escritos explícitamente pero se siguen de la ontología –por ejemplo, que una proposición sobre “desahucios” trata también, por herencia temática, de “vivienda- sin que haga falta repetir esa relación en cada dato.

2.5. GraphRAG

GraphRAG es la variante de RAG en la que la fuente de recuperación no es un almacén de fragmentos de texto sino un grafo de conocimiento consultado mediante un lenguaje de consulta estructurado (SPARQL en este trabajo). En lugar de recuperar pasajes de texto parecidos a la pregunta, un LLM traduce la pregunta en lenguaje natural a una consulta formal, el motor de grafos la ejecuta y devuelve filas de datos exactos, que un segundo LLM convierte en una respuesta narrada.

Esto resulta especialmente adecuado para preguntas de agregación y razonamiento multi-salto –“¿qué grupo ha presentado más proposiciones sobre vivienda?”, “¿cuántas se han aprobado por año?”– que un sistema de recuperación de texto libre no puede responder con precisión, porque exigen contar y cruzar datos de todo el corpus, no solo encontrar un pasaje parecido a la pregunta. Es, junto con el RAG vectorial tradicional, uno de los dos sistemas comparados en este TFG.

2.6. El Pleno del Ayuntamiento de Bilbao

Este trabajo se ha desarrollado sobre la documentación pública del Pleno del Ayuntamiento de Bilbao, el órgano superior de gobierno y control del municipio, integrado por la alcaldía y los concejales electos en cada mandato. En sus sesiones, ordinarias o extraordinarias, los distintos grupos políticos y la propia alcaldía presentan proposiciones que se debaten y se someten a votación; el resultado de esa votación –aprobada, rechazada, aprobada con enmienda o decaída– queda recogido, junto con el resto del debate, en el acta de la sesión.

El Ayuntamiento publica estas actas, junto con el resto de documentación de cada



Figura 3: Fachada del Ayuntamiento de Bilbao
Fuente: Wikimedia Commons, licencia CC0 (dominio público).

sesión (orden del día, extractos, resumen), en su portal de actas del Pleno, de acceso público. Es precisamente este archivo –de casi veinte años y 3.422 proposiciones– el que constituye el corpus documental de este trabajo; su descripción detallada, junto con el proceso que lo transforma en datos estructurados, se recoge en el Capítulo 7.

3. Descripción de requerimientos y análisis del estado del arte

El corpus documental de partida –las actas del Pleno– se describe en detalle, junto con su preprocesado, en el Capítulo 7.

3.1. Requisitos del sistema

3.2. Estado del arte de las tecnologías utilizadas

4. Planificación del proyecto

4.1. Diagrama EDT

La Figura 4 muestra la Estructura de Desglose del Trabajo (EDT) del proyecto, con los cinco paquetes de trabajo de primer nivel y sus subpaquetes, que se detallan en la Sección 4.2.

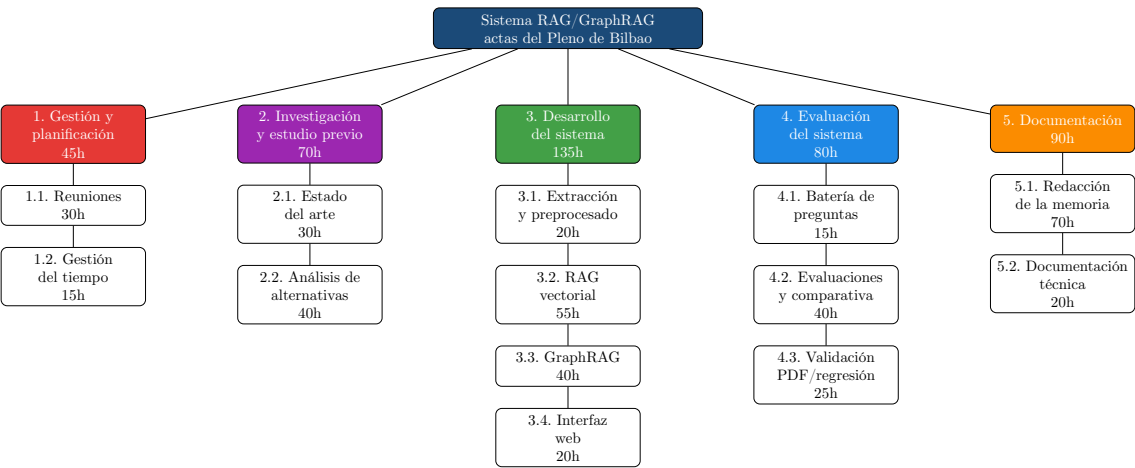


Figura 4: Diagrama EDT del proyecto

4.2. Descripción de tareas

En esta sección se describen los paquetes de trabajo mostrados en el diagrama EDT. Cada paquete se presenta en una tabla con su tiempo estimado, descripción, recursos empleados, predecesores y entregables –mismo formato en toda la memoria, un color por categoría de primer nivel del EDT.

1. Gestión y planificación	
Tiempo estimado	45 horas
Descripción	Tareas relacionadas con la planificación del proyecto y la gestión del tiempo con el director.
Recursos	Reuniones periódicas con el director, herramienta de diagramación.
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt, apuntes de seguimiento del trabajo.

Tabla 1: Paquete de trabajo: 1. Gestión y planificación

1.1. Reuniones con el director	
Tiempo estimado	30 horas
Descripción	Reuniones periódicas de seguimiento con el director para revisar el avance, resolver dudas técnicas y ajustar la planificación.
Recursos	Videollamada / presencial.
Predecesores	-
Entregables/salidas	Valoraciones del trabajo realizado y tareas siguientes acordadas.

Tabla 2: Paquete de trabajo: 1.1. Reuniones con el director

1.2. Gestión del tiempo	
Tiempo estimado	15 horas
Descripción	Planificación y seguimiento del tiempo dedicado a cada paquete de trabajo; elaboración de los diagramas EDT y Gantt.
Recursos	Herramienta de diagramación (Draw.io).
Predecesores	-
Entregables/salidas	Diagramas EDT y Gantt.

Tabla 3: Paquete de trabajo: 1.2. Gestión del tiempo

2. Investigación y estudio previo	
Tiempo estimado	70 horas
Descripción	Estudio de las técnicas y tecnologías en las que se apoya el sistema, y análisis de las principales alternativas tecnológicas antes de fijar la arquitectura.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	-
Entregables/salidas	Capítulos de Contexto y Análisis de alternativas de la memoria.

Tabla 4: Paquete de trabajo: 2. Investigación y estudio previo

2.1. Estudio del estado del arte	
Tiempo estimado	30 horas
Descripción	Estudio de LLM, RAG y sistemas GraphRAG existentes, y de trabajos similares sobre documentación administrativa.
Recursos	Documentación técnica y bibliografía de referencia.
Predecesores	-
Entregables/salidas	Capítulo de Contexto y estado del arte de la memoria.

Tabla 5: Paquete de trabajo: 2.1. Estudio del estado del arte

2.2. Análisis de alternativas tecnológicas	
Tiempo estimado	40 horas
Descripción	Comparación de LLM local frente a API en la nube, almacenamiento del grafo (rdflib/OWL-RL frente a una base de grafos dedicada) y estrategia de recuperación vectorial, incluyendo el diseño de la ontología OWL y el enfoque de razonamiento de GraphRAG.
Recursos	Documentación de las tecnologías evaluadas, pruebas exploratorias.
Predecesores	2.1. Estudio del estado del arte
Entregables/salidas	Capítulo de Análisis de alternativas de la memoria.

Tabla 6: Paquete de trabajo: 2.2. Análisis de alternativas tecnológicas

3. Desarrollo del sistema	
Tiempo estimado	135 horas
Descripción	Implementación del corpus estructurado, los dos sistemas de recuperación de información y la interfaz web.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Sistema completo: corpus estructurado, grafo de conocimiento, RAG vectorial, GraphRAG e interfaz web.

Tabla 7: Paquete de trabajo: 3. Desarrollo del sistema

3.1. Extracción y preprocesado del corpus	
Tiempo estimado	20 horas de trabajo activo
Descripción	Scraping del portal de actas, extracción de proposiciones y enriquecimiento temático mediante LLM (Capítulo 7). No incluye el tiempo de cómputo del propio enriquecimiento, que corre sin supervisión (ver Sección 4.5).
Recursos	Python, LangChain, API de Gemini.
Predecesores	2.2. Análisis de alternativas tecnológicas
Entregables/salidas	Corpus estructurado de proposiciones (<code>proposals_enriched.jsonl</code>).

Tabla 8: Paquete de trabajo: 3.1. Extracción y preprocesado del corpus

3.2. RAG vectorial	
Tiempo estimado	55 horas
Descripción	Implementación de la recuperación híbrida (semántica, literal y temática), reordenación de resultados y generación de respuestas con citación de fuentes.
Recursos	ChromaDB, LangChain, Cohere (reordenación), Groq/Ollama (generación).
Predecesores	3.1. Extracción y preprocesado del corpus
Entregables/salidas	Sistema RAG vectorial funcional.

Tabla 9: Paquete de trabajo: 3.2. RAG vectorial

3.3. GraphRAG	
Tiempo estimado	40 horas
Descripción	Construcción del grafo RDF/OWL, razonamiento automático, generación de SPARQL a partir de lenguaje natural y sanitización de los anti-patrones más frecuentes en las consultas generadas por el LLM.
Recursos	rdflib, owlrl, Ollama (generación de SPARQL y narración).
Predecesores	3.1. Extracción y preprocesado del corpus
Entregables/salidas	Sistema GraphRAG funcional (<code>bilbao_reasoned.ttl</code> + <code>graph_rag_sparql.py</code>).

Tabla 10: Paquete de trabajo: 3.3. GraphRAG

3.4. Interfaz web	
Tiempo estimado	20 horas
Descripción	Integración de ambos sistemas de recuperación en una interfaz web conversacional común, con citación de fuentes enlazada a la página exacta del PDF. Al construirse sobre Chainlit, un <i>framework</i> ya dado, el esfuerzo se centra en la integración de los dos <i>backends</i> , no en construir la interfaz desde cero.
Recursos	Chainlit.
Predecesores	3.2. RAG vectorial, 3.3. GraphRAG
Entregables/salidas	Interfaz web funcional.

Tabla 11: Paquete de trabajo: 3.4. Interfaz web

4. Evaluación del sistema	
Tiempo estimado	80 horas
Descripción	Diseño y ejecución de la evaluación comparativa de ambos sistemas, y validación de las respuestas contra las actas originales en PDF.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	3.4. Interfaz web
Entregables/salidas	Resultados de la evaluación comparativa (Capítulo 8).

Tabla 12: Paquete de trabajo: 4. Evaluación del sistema

4.1. Diseño de la batería de preguntas de referencia	
Tiempo estimado	15 horas
Descripción	Diseño de las preguntas de referencia y verificación manual de su respuesta correcta contra las actas originales.
Recursos	Lectura directa de las actas en PDF.
Predecesores	3.4. Interfaz web
Entregables/salidas	Batería de preguntas de referencia con respuesta verificada.

Tabla 13: Paquete de trabajo: 4.1. Diseño de la batería de preguntas de referencia

4.2. Evaluaciones y comparativa	
Tiempo estimado	40 horas
Descripción	Ejecución de la batería sobre ambos sistemas, análisis de la calidad de recuperación y de generación, y comparativa GraphRAG frente a RAG vectorial.
Recursos	Scripts de evaluación propios.
Predecesores	4.1. Diseño de la batería de preguntas de referencia
Entregables/salidas	Resultados comparativos entre ambos sistemas.

Tabla 14: Paquete de trabajo: 4.2. Evaluaciones y comparativa

4.3. Validación contra PDF real y regresión	
Tiempo estimado	25 horas
Descripción	Verificación manual del contenido de las respuestas contra el texto original de las actas (no solo contra la estructura de la respuesta), y ejecución de la suite de regresión ante cada cambio relevante.
Recursos	scripts/regression_qa.py, lectura directa de PDF.
Predecesores	4.2. Evaluaciones y comparativa
Entregables/salidas	Conjunto de casos verificados y corregidos (véase decisiones_tecnicas.md para el detalle de los casos encontrados).

Tabla 15: Paquete de trabajo: 4.3. Validación contra PDF real y regresión

4.2.1. Gestión del proyecto

4.2.2. Investigación y estudio previo

4.2.3. Desarrollo del sistema

4.2.4. Evaluación del sistema

4.2.5. Documentación

5. Documentación	
Tiempo estimado	90 horas
Descripción	Redacción de la memoria del TFG y de la documentación técnica del proyecto.
Recursos	Los recursos empleados se especifican en cada subpaquete.
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 16: Paquete de trabajo: 5. Documentación

5.1. Redacción de la memoria	
Tiempo estimado	70 horas
Descripción	Redacción de los capítulos de la memoria del TFG.
Recursos	L ^A T _E X (plantilla EHU).
Predecesores	4.3. Validación contra PDF real y regresión
Entregables/salidas	Memoria del TFG.

Tabla 17: Paquete de trabajo: 5.1. Redacción de la memoria

5.2. Documentación técnica	
Tiempo estimado	20 horas
Descripción	Documentación del razonamiento detrás de las decisiones técnicas no evidentes del sistema, separada del código (<code>decisiones_tecnicas.md</code>), y de los procedimientos de despliegue y uso (<i>READMEs</i> , <i>runbook</i>).
Recursos	Markdown.
Predecesores	- (paralelo al resto de paquetes)
Entregables/salidas	<code>decisiones_tecnicas.md</code> , <i>READMEs</i> y <i>runbook</i> del proyecto.

Tabla 18: Paquete de trabajo: 5.2. Documentación técnica

4.3. Gestión del tiempo

4.3.1. Estimación de horas frente a duración real

4.3.2. Diagrama de Gantt

4.4. Metodología de desarrollo

El desarrollo ha seguido un enfoque iterativo: cada componente se implementa, se prueba en vivo contra preguntas reales y se corrige antes de pasar al siguiente, en vez de implementar el sistema completo y evaluarlo solo al final. Cada iteración de corrección se apoya en dos verificaciones complementarias: una suite de regresión automática (`scripts/regression_qa.py`) que se relanza tras cualquier cambio relevante en la recuperación o la generación de SPARQL, y una validación manual contra el texto original de las actas en PDF –no solo contra la estructura de la respuesta o el propio sistema– para detectar fallos que la regresión automática no puede ver, como una mezcla de contenido entre dos proposiciones distintas dentro de un mismo bloque bien formado. Este proceso, con sus hallazgos y las correcciones aplicadas en cada ronda, queda documentado en `decisiones_tecnicas.md`, que sirve además de base directa para la Sección 6 de esta memoria.

4.5. Evaluación económica

4.5.1. Costes por mano de obra

El coste de mano de obra se calcula a partir del total de horas estimadas en el diagrama EDT (Sección 4.2), valoradas a una tarifa de referencia de ingeniero junior/becario en prácticas.

Concepto	Valor
Horas totales estimadas	420 h
Tarifa/hora	15 €/h
Coste de mano de obra	6.300 €

Tabla 19: Coste de mano de obra

4.5.2. Costes en hardware

El proyecto se ha desarrollado con dos equipos propios: un portátil, usado para la práctica totalidad del trabajo activo (desarrollo, redacción, reuniones), y un ordenador de sobremesa, usado únicamente para el cómputo sin supervisión que no requiere intervención activa –el enriquecimiento LLM local del corpus completo y el estudio de comparación de modelos para la generación de SPARQL (Sección 2.5)–. Ambos equipos se amortizan de forma proporcional a las horas dedicadas al proyecto sobre una vida útil estimada de 4 años (~7.000 horas de uso).

Equipo	Precio	Horas usadas	Vida útil	Coste
Portátil (Huawei, Ryzen 5)	650 €	420 h	7.000 h	39 €
Ordenador de sobremesa ¹	900 €	65 h	7.000 h	8 €
Coste de hardware				47 €

Tabla 20: Coste de hardware (amortizado)

4.5.3. Costes en software

Todas las herramientas de software empleadas son de código abierto o se han usado en su capa gratuita (ChromaDB, LangChain, Chainlit, Ollama, Groq y el plan de prueba de Cohere), salvo el enriquecimiento temático del corpus con la API de pago de Gemini, cuyo coste real quedó medido y documentado durante el desarrollo.

Servicio	Coste
Groq (capa gratuita)	0 €
Cohere (plan de prueba)	0 €
Ollama, ChromaDB, LangChain, Chainlit (código abierto)	0 €
API de Gemini (enriquecimiento del corpus)	7 €
Coste de software	7 €

Tabla 21: Coste de software

¹Precio y horas de uso estimados; especificaciones exactas pendientes de confirmar.

4.5.4. Costes indirectos

Se considera como coste indirecto el consumo eléctrico de ambos equipos durante las horas dedicadas al proyecto, a una tarifa media residencial de referencia de 0,20 €/kWh.

Equipo	Consumo	Horas	Coste
Portátil	~50 W	420 h	4 €
Ordenador de sobremesa	~300 W	65 h	4 €
Coste indirecto (electricidad)			8 €

Tabla 22: Costes indirectos (electricidad)

4.5.5. Coste total

Concepto	Coste
Mano de obra	6.300 €
Hardware (amortizado)	47 €
Software	7 €
Indirectos (electricidad)	8 €
Coste total	6.362 €

Tabla 23: Coste total del proyecto

El coste de mano de obra domina claramente el presupuesto (~99 % del total), lo cual es coherente con un proyecto que no requiere hardware especializado ni servicios de pago más allá de una única pasada puntual de enriquecimiento del corpus.

4.6. Análisis de riesgos

4.6.1. Medidas de prevención y contingencia

1. Dependencia de APIs externas (Groq)	
Descripción	El sistema usa la API de Groq para generar las respuestas finales; si el servicio falla o se agota la cuota diaria, la calidad de la respuesta puede degradarse.
Plan de prevención	Vigilar el consumo de cuota antes de lanzar baterías de pruebas extensas.
Plan de contingencia	<i>Fallback</i> automático a un modelo local vía Ollama cuando Groq no responde o se agota la cuota.
Probabilidad	Media
Impacto	Bajo (el sistema sigue funcionando, con menor calidad de redacción)

Tabla 24: Riesgo: 1. Dependencia de APIs externas

5. Análisis de alternativas

5.1. Modelos de lenguaje: local (Ollama) frente a API en la nube (Groq / Gemini)

5.1.1. Puntos positivos

5.1.2. Puntos negativos

5.1.3. Conclusión

5.2. Almacenamiento del grafo: rdflib/OWL-RL frente a una base de grafos dedicada (p.ej. GraphDB, Neo4j)

5.2.1. Puntos positivos

5.2.2. Puntos negativos

5.2.3. Conclusión

5.3. Almacén vectorial y recuperación: ChromaDB e híbrida (semántica + literal + temática) frente a solo semántica

5.3.1. Puntos positivos

5.3.2. Puntos negativos

5.3.3. Conclusión

6. Descripción de la solución propuesta

El proceso de obtención y preparación del corpus sobre el que opera el sistema se describe aparte, en el Capítulo 7.

6.1. Arquitectura general del sistema

6.2. Construcción del grafo de conocimiento

6.3. Sistema de recuperación de información

6.3.1. RAG vectorial

6.3.2. GraphRAG

6.4. Generación de respuestas

6.5. Interfaz web del sistema

6.6. Flujo completo del sistema

6.7. Configuración y decisiones de implementación

6.7.1. Temperatura y parámetros de los LLM

6.7.2. Gestión de cuotas y memoria

6.7.3. Construcción del *prompt* para generación

7. Corpus y preprocesado de datos

7.1. El corpus de actas del Pleno

7.2. Pipeline de preprocesado

7.2.1. Descarga automatizada de las actas

7.2.2. Extracción de proposiciones

7.2.3. Enriquecimiento con un LLM

8. Análisis de resultados

8.1. Objetivo de la evaluación

8.2. Metodología

Categoría de pregunta	GraphRAG	Vectorial
Conteo simple	6	–
Ratio / comparación de grupos	4	–
Ranking	3	–
Consulta narrativa / contexto abierto	–	13

Tabla 25: Categorías de preguntas de la batería de referencia (ejemplo)

8.3. Análisis sobre la recuperación

8.4. Análisis sobre la generación

8.5. Comparativa GraphRAG frente a RAG vectorial

8.6. Configuración final y valoración global

8.7. Limitaciones

9. Conclusiones y trabajo futuro

9.1. Conclusiones del trabajo

9.2. Cumplimiento de objetivos

9.3. Conclusiones sobre las pruebas realizadas

9.4. Contribuciones científico-técnicas

9.5. Trabajo futuro

Bibliografía

[1] Autores. *Título*. Editorial, Año.

[2] Autores. *Título*. Editorial, Año. <https://www.example.com>